

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра информационной безопасности

ОТЧЁТ

по лабораторной работе №1

по дисциплине: «Компьютерная графика»

Тема:

«Отсечение произвольного многоугольника выпуклым окном»

Студент гр. 4364	_____	Ефремов Э. А.
Студент гр. 4364	_____	Поляков Н. Н
Преподаватель	_____	Матвеева И. В.

ЗАДАНИЕ

Разработать программный продукт, реализующий отсечение произвольного многоугольника выпуклым окном. Предусмотреть: ввод координат вершин отсекаемого многоугольника, ввод координат вершин выпуклого окна,

ЦЕЛЬ РАБОТЫ

Изучить математический аппарат отсечения многоугольников, освоить алгоритм последовательного отсечения произвольного многоугольника сторонами выпуклого окна, реализовать соответствующий алгоритм и веб-интерфейс для интерактивного управления параметрами.

1 Представление многоугольников

Многоугольник задаётся списком вершин (x_i, y_i) , соединённых рёбрами. Окно также является многоугольником и должно быть выпуклым. Отсечение выполняется последовательно каждой стороной окна.

В программе используются два списка точек:

- `subject` — отсекаемый многоугольник;
- `clip` — выпуклое окно.

2 Математическая основа

2.1 Проверка выпуклости окна

Для каждой тройки последовательных вершин O, A, B вычисляется векторное произведение:

$$\text{vec}(O, A, B) = (A_x - O_x)(B_y - O_y) - (A_y - O_y)(B_x - O_x).$$

Если все ненулевые значения `vec` имеют одинаковый знак, многоугольник выпуклый. Дополнительно проверяется отсутствие дублирующихся вершин.

2.2 Положение точки относительно стороны окна

Пусть сторона окна задана точками a и b , а c — центр окна. Точка P считается внутренней для этой стороны, если

$$\text{vec}(a, b, P) \cdot \text{vec}(a, b, c) \geq 0.$$

Это условие используется в программной реализации.

2.3 Пересечение отрезка со стороной окна

Для отрезка P_1P_2 и стороны AB точка пересечения ищется через параметр t :

$$P = P_1 + t(P_2 - P_1), \quad P = A + u(B - A).$$

В коде вычисляется знаменатель

$$\text{denom} = (x_1 - x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 - x_4),$$

и параметр

$$t = \frac{(x_1 - x_3)(y_3 - y_4) - (y_1 - y_3)(x_3 - x_4)}{\text{denom}}.$$

Если $\text{denom} \approx 0$, отрезки параллельны, и возвращается конечная точка P_2 .

2.4 Алгоритм Сазерленда–Ходжмана

Алгоритм последовательно отсекает многоугольник каждой стороной выпуклого окна. Для каждой стороны окна формируется новый список вершин. Для каждой пары соседних вершин S (предыдущая) и E (текущая) возможны четыре случая:

1. S и E вне окна — ничего не добавляется;
2. S вне, E внутри — добавляется точка пересечения и вершина E ;
3. S и E внутри — добавляется только вершина E ;
4. S внутри, E вне — добавляется только точка пересечения.

После обработки всех сторон окна получается отсечённый многоугольник.

3 Средства реализации

Программа реализована на языке Python. Используются стандартные библиотеки:

- `http.server` — веб-сервер;
- `urllib.parse` — разбор POST-параметров;
- `webbrowser` — автоматическое открытие браузера;
- `threading` — запуск браузера через таймер.

Визуализация выполнена средствами HTML и SVG. Специализированные графические библиотеки типа OpenGL не применялись.

4 Структура программы

Основные функции:

- `cross(o, a, b)` — вычисление векторного произведения;
- `is_convex(poly)` — проверка выпуклости многоугольника;
- `centroid(poly)` — вычисление центра многоугольника;
- `line_intersection(p1, p2, a, b)` — пересечение отрезка со стороной окна;

- `sutherland_hodgman(subject, clip)` — основной алгоритм отсечения;
- `parse_points(text)` — разбор координат из текста;
- `render_page(...)` — формирование HTML-страницы с SVG;
- класс `Handler` — обработка GET- и POST-запросов.

Ключевой фрагмент алгоритма:

```

1 def sutherland_hodgman(subject, clip):
2     if not is_convex(clip):
3         return None
4     center = centroid(clip)
5     output = list(subject)
6     n = len(clip)
7     for i in range(n):
8         a = clip[i]
9         b = clip[(i + 1) % n]
10        input_list = output
11        output = []
12        if not input_list:
13            break
14        S = input_list[-1]
15        for E in input_list:
16            c_S = cross(a, b, S) * cross(a, b, center)
17            c_E = cross(a, b, E) * cross(a, b, center)
18            S_in = c_S >= 0
19            E_in = c_E >= 0
20            if E_in:
21                if not S_in:
22                    output.append(line_intersection(S, E, a, b)
23                    )
24                    output.append(E)
25                elif S_in:
26                    output.append(line_intersection(S, E, a, b))
27                S = E
28        return output

```

5 Интерфейс пользователя

Веб-страница содержит:

- SVG-поле размером 800×600 для отображения сцены;
- исходный многоугольник отображается красным цветом;
- окно отображается белым пунктиром;

- отсечённый многоугольник отображается зелёным цветом;
- текстовые области для ввода координат многоугольника и окна;
- кнопку «Пересчитать»;
- легенду и блок вывода ошибок.

Сервер запускается на `127.0.0.1:8080` и автоматически открывает браузер. При GET-запросе выводятся значения по умолчанию. При POST-запросе координаты считываются из формы, проверяются и передаются в алгоритм.

6 Алгоритм работы программы

1. Пользователь открывает страницу в браузере.
2. При GET-запросе отображаются многоугольник и окно по умолчанию.
3. Пользователь может изменить координаты в текстовых полях.
4. При нажатии кнопки «Пересчитать» выполняется POST-запрос.
5. Программа проверяет:
 - в многоугольнике не менее 3 точек;
 - в окне не менее 3 точек;
 - в окне нет дублирующихся точек;
 - окно выпуклое.
6. Если проверки пройдены, вызывается `sutherland_hodgman`.
7. Результат отображается на SVG-поле.
8. При ошибке выводится соответствующее сообщение.

7 Тестирование

Были проведены следующие проверки:

1. Значения по умолчанию: многоугольник из 8 вершин и выпуклое окно из 5 вершин. Программа корректно строит отсечённый контур.
2. Окно с дублирующимися точками — выводится сообщение об ошибке.

3. Невыпуклое окно — выводится сообщение об ошибке.
4. Многоугольник или окно с менее чем 3 точками — выводится сообщение об ошибке.
5. Изменение координат через форму — результат пересчитывается.
6. Проверка обработки замыкающей стороны многоугольника — алгоритм корректно замыкает контур.

Пример значений по умолчанию:

- многоугольник:

100,300
250,100
400,250
550,100
700,300
550,500
400,350
250,500

- окно:

250,150
550,180
600,400
400,500
200,400

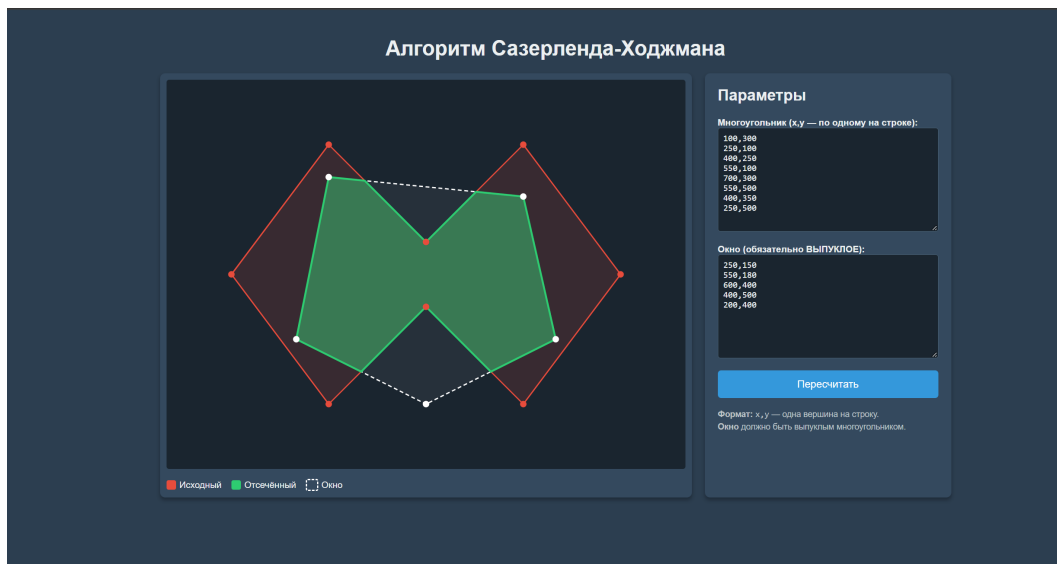


Рис. 1: Интерфейс программы с отсечением многоугольника

ВЫВОДЫ

В ходе лабораторной работы был изучен алгоритм Сазерленда–Ходжмана для отсечения произвольного многоугольника выпуклым окном. Рассмотрены математические основы: проверка выпуклости, определение положения точки относительно стороны окна, вычисление точек пересечения отрезков. Разработан программный продукт на языке Python с веб-интерфейсом, который принимает координаты многоугольника и окна, проверяет корректность окна, и выполняет отсечение.